



THE COMPLETE OWNER'S MANUAL

The Recruit AI Book

Everything about the recruitment automation platform: what it does and who it's for, how it was designed and built, how to set it up from zero, how to run it every day, and how to hand it to a paying client.

6 modules

2 human approval gates

Multi-tenant

88 automated checks

Built and documented by Claude for Mayank · 2026

What Recruit AI is

Recruit AI is a **multi-tenant recruitment automation platform**. One installation serves many recruiting organizations; each organization's roles, candidates, emails, and reports are invisible to every other. AI does the heavy reading and writing — scoring resumes, ranking talent pools, drafting personalized emails — while humans keep control of every moment that touches a candidate or a client.

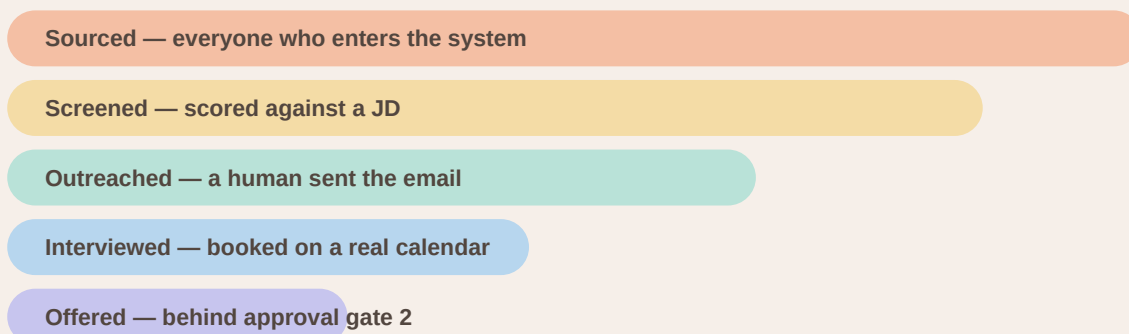
THE TWO NON-NEGOTIABLES (ENFORCED IN CODE, NOT POLICY)

1. No email ever reaches a candidate or client without an explicit human click. The AI only writes drafts.
2. Moving anyone to the offer or closed stage requires a recorded human approval — even when the change comes from an external ATS.

The six modules

MODULE	WHAT IT DOES
Sourcing	CSV / paste import of candidate lists (e.g. exported LinkedIn searches), an org-wide talent pool that persists across roles, an AI Boolean-search generator, and automatic duplicate detection.
Screening & Scoring	Resume PDFs and pool profiles scored 0–100 against a job description, with sub-scores, a written rationale, and a ranked shortlist with bulk approve/reject.
Engagement	AI-drafted personalized outreach, status updates on stage changes, and no-reply follow-ups — all queued in an Outbox for review before a human sends them.
Scheduling	Google Calendar per recruiter: slots proposed from real availability, a no-login booking link for candidates, automatic Meet links, 24-hour reminder drafts, 48-hour feedback nudges.
Data & ATS Sync	Generic signed webhooks in both directions — works with any webhook-capable ATS (Greenhouse/Lever style) — plus fuzzy duplicate merging on import.
Reporting	A live hiring funnel with drop-off rates, a detailed recruiter dashboard, a PII-free client view, and weekly summaries stored and downloadable as PDF.

The pipeline at a glance



Closed

Use cases & playbooks

Who it's for

- **Recruiting agencies** placing candidates for client companies — each engagement gets clean funnel reporting, and the client-facing dashboard shares progress without exposing candidate identities.
- **In-house talent teams** that want AI leverage on screening volume without surrendering the candidate relationship to automation.
- **Staffing firms** sitting on years of spreadsheets — the talent pool turns old sourcing lists into a searchable, re-scorable asset.

Playbook: fill a backend role this month

1. Create the role and paste the full job description — every score derives from it.
2. Import your exported LinkedIn search as CSV into the Talent Pool, and upload any direct-applicant resume PDFs on the role page.
3. Click **Match from talent pool** — everyone you've ever sourced gets ranked against this JD in minutes.
4. Sweep the shortlist: bulk-approve the top band, reject the clear noes.
5. Draft outreach for each approved candidate; polish and send from the Outbox in one sitting.
6. After four days, click **Draft follow-ups** — silent threads get a polite nudge, written for you.
7. For candidates who reply, propose interview slots — they self-book from your live calendar, and the Meet invite handles itself.
8. Log feedback after each interview (the system nudges the interviewer if you forget), approve for offer, close, and pull the weekly PDF for the hiring manager.

Playbook: reactivate a dead spreadsheet

1. Export the old tracker to CSV — the importer maps flexible headers (Name/Full Name, YoE/Experience, Employer/Company...) and folds unknown columns into the profile text so nothing is lost.
2. Import. Duplicates merge automatically on email, or on near-identical names corroborated by phone/company; ambiguous pairs are flagged for human review, never auto-merged.
3. Open any live role and match the pool against it — dormant candidates often rank surprisingly well against new JDs.

Playbook: the Friday client report

1. Open Reports and flip to **Client view** — aggregate funnel, drop-offs, and role names only; candidate PII is excluded by construction, so it's safe to screen-share.
2. Download the weekly PDF (auto-generated every week; stored, never auto-emailed) and attach it to your own client email.

How it was built

The platform was built by Claude (Anthropic's AI) in six phases over a working session with Mayank, each phase ending with a hard verification gate: an automated script proving the phase's promise against real services — real database, real AI calls, a real email in a real inbox, a real event on a real Google Calendar. No phase advanced until its gate passed. Those scripts remain in `scripts/verify_phase1-6.py` and can be re-run anytime.

The stack

LAYER	TECHNOLOGY	WHY
Frontend	React (Vite) + Tailwind v4	Fast dev loop; the pastel design system lives in ~40 lines of theme tokens.
Backend	FastAPI (Python)	One router per module; typed request models; trivially deployable.
Database + Auth	Supabase (Postgres + GoTrue)	Row Level Security gives real multi-tenancy at the database layer.
AI	Gemini (2.5 flash family)	Structured-output JSON for scoring and drafting; env-switchable models with fallback.
Email	Resend	One clean REST call; sandbox mode for safe development.
Calendar	Google Calendar API	Free/busy for slot proposals; events with Meet links; OAuth per recruiter.
PDF	fpdf2	Weekly summaries rendered on demand.

The multi-tenancy design

Every data table carries an organization id and a Row Level Security policy: a member of one organization physically cannot read another's rows, even talking to the database directly. The backend uses a service key (which bypasses RLS) and therefore scopes every single query by the organization resolved from the caller's verified JWT — a belt-and-braces design verified by a test that tries, and fails, to cross the boundary both through the API and through the database itself.

The approval gates as code

Gate 1: outreach can only be drafted for shortlist-approved candidates, and the only code path that sends an email is the explicit send endpoint behind the Outbox button. Gate 2: the offer/closed stages reject any transition — from the UI or from an inbound ATS webhook — until a recruiter has recorded an offer approval. The single automatic email in the whole system is the internal feedback nudge to the interviewer, which never goes to a candidate or client.

Deliberate boundaries

- **No LinkedIn scraping.** It violates LinkedIn's terms and risks account bans; sourcing works via CSV export/import, with a clean integration point stubbed for the partner-only Recruiter API.

- **WhatsApp is stubbed.** The official Business Platform API requires Meta verification; engagement is email-first behind an interface WhatsApp can slot into later.
- **Workday flagged, not faked.** The ATS layer speaks generic webhooks; Workday's enterprise API tier is a commercial arrangement, not a code problem.

Real bugs found by the verification gates

BUG	FOUND BY	FIX
Gemini client garbage-collected mid-request	Phase 2 batch scoring failing	Singleton client
Free-tier model quotas are per-model	Repeated 429/503 during verification	Retry + env-switchable primary/fallback models
strip(" at") ate letters — "Zeta" became "Ze"	Reading ranked output line by line	Proper join + regression check
Batch scoring silently skipped candidates	Assertion that all 20 imports must score	Straggler re-scoring passes
Supabase free tier drops idle connections	Intermittent one-off 500s	Retrying transport, keep-warm ping, client GET retry

That table is the honest argument for verification-driven building: every one of these would otherwise have surfaced in front of a client.

Setting it up from zero

Four free-tier accounts, two env files, six migrations, two processes. Roughly an hour the first time.

1 • Accounts and keys

SERVICE	WHAT YOU NEED	WHERE
Supabase	A project; its URL, secret key (backend only), publishable key (frontend)	supabase.com -> Project Settings -> API Keys
Google AI Studio	A Gemini API key	aistudio.google.com/apikey
Resend	An API key (sandbox is fine to start)	resend.com -> API Keys
Google Cloud	OAuth web client (ID + secret), Calendar API enabled, redirect URI <code>http://localhost:8000/api/calendar/oauth/callback</code>	console.cloud.google.com -> Credentials

2 • Configuration

Copy `backend/.env.example` -> `backend/.env` and fill in the keys above. Copy `frontend/.env.example` -> `frontend/.env` with the Supabase URL and publishable key. **The secret key must never appear in the frontend.**

3 • Database

Run each file in `supabase/migrations/` (0001 -> 0006, in order) in the Supabase SQL editor. The optional `0000_optional_admin_exec.sql` installs a service-role-only RPC so future migrations can be applied programmatically — convenient, with the trade-off documented in the file.

4 • Run it

```
# terminal 1 – backend
cd backend
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python -m uvicorn app.main:app --port 8000

# terminal 2 – frontend
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173`, sign up, and your organization exists.

5 • Prove it works

```
backend\.venv\Scripts\python scripts\verify_phase1.py # tenant isolation
backend\.venv\Scripts\python scripts\verify_phase2.py # import + AI ranking
backend\.venv\Scripts\python scripts\verify_phase5.py # gates + webhooks
backend\.venv\Scripts\python scripts\verify_phase6.py # reporting + PDF
```

(Phases 3 and 4 also have suites, but they send a real email and book a real calendar event — run them deliberately.)

The operating manual

Daily rhythm

1. **Roles** — create a role with its full JD; open it to work a pipeline.
2. **Add candidates** — resume PDFs on the role page, CSVs on the Talent Pool page, or Match from talent pool to score everyone you already have.
3. **Shortlist** — approve/reject (singly or in bulk). Approval is gate 1: it contacts no one, it only unlocks contact.
4. **Outbox** — every AI-drafted email waits here. Edit inline, then Send or Discard. Set the follow-up window and click Draft follow-ups for silent threads; mark replied stops a sequence.
5. **Schedule** — on an approved candidate: duration, optional hiring-manager email, Propose slots + draft email. Send the drafted link; the candidate self-books; the Meet invite is automatic.
6. **Interviews** — statuses, Meet links, feedback logging. Reminder drafts appear 24h before; interviewers get nudged 48h after if feedback is missing; Run checks now forces a pass.
7. **Offer** — click Approve for offer (gate 2), then move the stage to offer/closed.
8. **Reports** — funnel with drop-offs, recruiter/client view toggle, weekly PDF downloads.

Settings, once per recruiter

- **Google Calendar** — connect the account whose calendar interviews should land on.
- **ATS sync** — paste the ATS's webhook URL and a shared secret; give the ATS your displayed inbound URL. Test with Send test event; watch the live log below it.

MENTAL MODEL

The AI is a tireless junior recruiter who reads everything and drafts everything, but owns no Send button, no offer, and no client relationship. Those are yours.

Limits & troubleshooting

Known limits (by design or by tier)

- **Resend sandbox** delivers only to the Resend account owner's inbox until you verify a sending domain — perfect for testing, useless for real candidates. Verify a domain before go-live.
- **Gemini free tier** has per-model daily quotas; heavy scoring days can exhaust one. The app falls back automatically, and GEMINI_MODEL / GEMINI_FALLBACK_MODEL in backend/.env switch models without code changes. Billing removes the ceiling.
- **Supabase free tier** occasionally drops idle connections or responds slowly. The app retries transparently; a persistent outage is upstream, not local.
- **Unverified Google OAuth app** shows a warning screen during calendar connect (Advanced -> Go to app). Google's verification process removes it.
- **No LinkedIn scraping, WhatsApp stubbed, Workday commercial** — see Chapter 3.

Symptom -> fix

SYMPTOM	FIX
"Invalid login credentials"	Password is wrong; an admin resets it in Supabase -> Authentication -> Users. (Self-serve reset is a sensible next feature.)
"Email not confirmed" at sign-in	Set the Site URL in Supabase Auth settings so confirmation links resolve, or confirm the user from the Supabase dashboard.
Scoring "temporarily unavailable"	Quota or load spike — wait, switch models via env, or enable billing.
Calendar connect -> redirect_uri_mismatch	The redirect URI isn't registered on the OAuth client (Authorized redirect URIs — not the JavaScript-origins box).
Calendar calls -> 403	Enable the Google Calendar API for the project.
Backend won't start: port in use	An old process holds port 8000 — kill it and relaunch.

Delivering it to a client

Choose the shape

- **One shared installation, one organization per client** — the multi-tenant design exists exactly for this. Cheapest to run; you operate it as a service.
- **Dedicated installation per client** — their own Supabase project and keys. More setup, total data separation, easiest story for security-sensitive clients.

Production checklist

1. **Fresh credentials.** New Supabase project (or at minimum rotated keys), new Gemini key, new Resend key, new Google OAuth client. Never reuse development keys that have been shared in chats or screenshots.
2. **Verify a sending domain in Resend** (e.g. talent.clientname.com) and set EMAIL_FROM — this is what makes candidate email real.
3. **Deploy the backend** — a single FastAPI service; Render, Railway, or Fly all take it with uvicorn and the env vars. The in-process scheduler travels with it.
4. **Deploy the frontend** — npm run build produces a static bundle for Vercel/Netlify; point its API proxy at the backend URL and put both behind the client's domain with HTTPS.
5. **Update Google OAuth** — add the production callback URL to the OAuth client; submit for verification to remove the warning screen.
6. **Supabase production settings** — Site URL set to the real domain (fixes confirmation emails); enable daily backups.
7. **Gemini billing on** — free-tier quotas are for building, not for a client's Monday morning.
8. **Run the verification suites against production once** — they are your acceptance test, and their green output is a deliverable you can literally show the client.

Onboarding the client (a one-hour session)

1. Create their organization and accounts; each recruiter connects their Google Calendar (Settings).
2. Configure ATS webhooks together if they have one, and send the test event while you're both watching the log.
3. Walk one real role through the whole pipeline live: import -> rank -> approve -> send one outreach -> book one interview. The gates sell themselves.
4. Hand over the operating manual (Chapter 5, or docs/USER_GUIDE.md in the repo) and agree on who owns the Outbox daily.

The handoff pack

- This book, plus the in-repo README.md and docs/USER_GUIDE.md.
- A credentials sheet delivered through a password manager — never email or chat.

- The green verification-suite output from their production environment.
- A support agreement: what you monitor, response times, and the upgrade path (WhatsApp once Meta-verified, LinkedIn partner API if they have Recruiter, an n8n workflow layer for no-code cadence editing).

THE PITCH, IN ONE SENTENCE

Your recruiters keep every decision that matters — the AI just makes them ten times faster at everything in between.